ARTE DE AMAR

DOCUMENTAÇÃO DO SISTEMA

Plano e relatório de *testes*

Estratégia em caixa branca, cinza e preta, rastreabilidade e achados.

Site e painel da OSC Arte de Amar
Palmas, Tocantins · Projeto Integrador 1 · Sistemas de Informação · UNITINS

Versão 1.2.0
11 de novembro de 2026

ARTE DE AMAR

Plano e relatório de testes

Estratégia em caixa branca, cinza e preta, rastreabilidade e achados.

Como o sistema é verificado, o que cada suíte cobre e como cada requisito de requisitos.md é testado.

Resumo

suíte	técnica	o que roda	testes
Unitários	caixa branca	bibliotecas da API e do site, isoladas	122
Banco	caixa branca	migrations aplicadas do zero num SQLite em memória	34
API	caixa preta e caixa cinza	a API real (wrangler, runtime da Cloudflare) com um D1 isolado	155
E2E	caixa preta	site e painel num navegador, no computador (1440px) e no celular (390px)	48
Acessibilidade	caixa preta	axe-core, regras WCAG 2 A e AA, dentro do E2E	incluída no E2E
Total			**359**

Cobertura de linhas dos unitários: 89% nas bibliotecas (`functions/_lib` , `src/lib` , zip do painel). O que fica de fora do unitário (`auditoria.ts` e `conteudo.ts`) depende do banco e é coberto pela suíte de API.

As três técnicas

Caixa branca conhece o código e testa os caminhos de dentro: cada ramo da validação, o hash de senha, a assinatura do token, a troca do conteúdo de reserva, o formato binário do zip. Fica em `tests/unit/` e `tests/db/` .

Caixa preta só conhece a interface: manda um pedido HTTP ou clica na tela e confere a resposta, sem saber como foi feito. É a maior parte de `tests/api/` e todo o `tests/e2e/` . Os contratos em `tests/contratos.ts` (schemas Zod) definem o que a API promete ao site e ao painel.

Caixa cinza faz o pedido pela interface, como a preta, mas depois abre o banco do servidor de teste (só leitura) e confere o que foi gravado: que a senha virou hash, que a auditoria registrou quem fez, que a versão subiu, que o dado inválido não chegou ao banco. Os testes marcados "caixa cinza" em `tests/api/` fazem isso pela função `banco()` de `tests/api/cliente.ts` .

Ambiente

Nenhum teste toca o banco de desenvolvimento nem o de produção. `tests/servidor-teste.mjs` cria, a cada rodada, uma pasta temporária, aplica todas as migrations, cria um usuário de teste e sobe o `wrangler pages dev` apontado para ela. Os testes de API e os E2E usam esse servidor; ao terminar, a pasta é apagada.

Os arquivos da suíte de API contam uma história em ordem (01-contrato até 09-seguranca) sobre um banco só, por isso rodam em sequência, pelo nome.

Como rodar

```
npm run build      # a API e os E2E servem o dist/
npm test           # unitários + banco + API
npm run test:e2e   # E2E e acessibilidade (baixe o navegador uma vez: npx playwright install chromium)
npm run test:tudo  # tudo, na ordem
npm run test:cobertura
```

No CI (`.github/workflows/testes.yml`) a pirâmide inteira roda a cada push e pull request na `main`. Se o E2E falhar, o relatório do Playwright, com trace e screenshot do erro, fica salvo como artefato.

O que cada suíte testa

Unitários (`tests/unit/`)

arquivo	foco
`auth.test.ts`	hash PBKDF2 (salt, compatibilidade com o script de admin), JWT (adulterado, outro segredo, expirado em 8 h, malformado), flags do cookie
`entidades.test.ts`	lista fechada de tabelas, cada regra de validação, a regra da meta, campos alterados para a auditoria
`http.test.ts`	leitura de JSON (tipo, tamanho, formato), formato único de erro, erro interno sem vazar detalhe, ids de rota
`textos.test.ts`	chaves únicas, texto original dentro do limite, aplicação no site sem mexer no catálogo de fotos
`conteudo-remoto.test.ts`	troca do conteúdo no lugar, layout derivado da ordem, reserva com API em erro, fora do ar, lenta (2,5 s) ou fora do contrato

arquivo	foco
`pix.test.ts`	CRC16 com o valor de verificação padrão, o payload do manual do Pix do Banco Central reproduzido caractere por caractere, normalização das chaves (CNPJ, CPF, telefone, e-mail, aleatória), limites de nome e cidade
`utilitarios.test.ts`	zip relido byte a byte e conferido com o CRC-32 do zlib (e pelo Python, quando instalado), nomes de arquivo, senha temporária (formato, sem caracteres ambíguos, sem repetição)

Banco (`tests/db/schema.test.ts`)

Migrations em sequência e aplicáveis do zero, `integrity_check`, seed, e o que o banco garante sozinho mesmo se a API errar: CHECKs de status e valores, e-mail único sem diferenciar maiúsculas, foto em uso protegida por chave estrangeira, retrato apagado vira "sem foto" sem apagar a pessoa, versão inicial e UPDATE condicional, índices usados pela consulta da auditoria.

API (`tests/api/`)

arquivo	foco
`01-contrato`	`/api/conteudo` dentro do contrato, seed, nada interno exposto, formato único de erro
`02-autenticacao`	login, mesma mensagem para e-mail inexistente e senha errada, trava depois de 5 erros, cookie, logout, as 26 rotas do painel exigindo login
`03-cadastros`	criar, editar, ocultar, reordenar e apagar refletindo no site; validação; rotas e ids com injeção de SQL; configurações
`04-fotos`	upload de JPEG, PNG e WebP reais; tipo pelos bytes; SVG, arquivo disfarçado e arquivo grande recusados; download; proteção de foto em uso
`05-voluntarios`	inscrição, normalização do telefone, campo-isca, limite por IP, status, apagar arquivados de uma vez
`06-textos`	editar, foto de seção, voltar ao original, normalização, limites, pacote inválido não grava pela metade
`07-concorrencia`	409 com o nome de quem salvou antes, dez gravações simultâneas com exatamente uma vencendo, itens diferentes ao mesmo tempo
`08-usuarios-auditoria`	senha temporária, troca obrigatória, sessão derrubada ao tirar acesso, auditoria por período, espaço do banco
`09-seguranca`	CSRF, XSS armazenado, corpo grande, tipo errado, prototype pollution, método não suportado, erro sem detalhe interno

E2E (`tests/e2e/`)

arquivo	foco
`site.spec.ts`	site sem erro de console nem rolagem lateral; reserva com API fora e lenta; inscrição chegando ao painel; erro no formulário; WhatsApp quando a API cai; HTML de texto editado não executa; seção vazia some; QR do PIX lido do print da tela (jsQR) igual ao copia e cola; chave trocada no painel troca o QR
`painel.spec.ts`	login, editar meta e ver no site, editar e voltar texto ao original, upload real (foto de 3000px chega com 1600px), duas pessoas no mesmo projeto, dar acesso e trocar senha obrigatória, auditoria, WhatsApp marcando como chamado, todas as abas no celular
`acessibilidade.spec.ts`	axe-core no site, no login e em sete abas do painel

Rastreabilidade

Cada teste cita no nome o requisito que verifica. Contagem de testes (ou grupos parametrizados) por requisito:

requisito	unit	banco	API	E2E
RF01 conteúdo do banco no site	✓	✓	✓	✓
RF02 ocultos e seção vazia	✓		✓	✓
RF03 conteúdo de reserva	✓			✓
RF04 inscrição de voluntário	✓		✓	✓
RF05 WhatsApp quando a API cai	✓			✓
RF06 textos editados no site	✓		✓	✓
RF07 WhatsApp e PIX configuráveis	✓		✓	
RF21 QR Code e Pix copia e cola	✓			✓
RF10 login e sessão			✓	✓
RF11 cadastros	✓		✓	✓
RF12 fotos	✓		✓	✓
RF13 redução e WebP				✓
RF14 textos e voltar ao original	✓		✓	✓
RF15 voluntários no painel			✓	✓
RF16 configurações e senha			✓	
RF17 acessos e senha temporária	✓		✓	✓

requisito	unit	banco	API	E2E
RF18 troca obrigatória			✓	✓
RF19 auditoria	✓		✓	✓
RF20 espaço do banco			✓	✓
RNF01 hash de senha	✓		✓	
RNF02 sessão e cookie	✓		✓	
RNF03 CSRF			✓	
RNF04 trava de login			✓	
RNF05 SQL injection	✓		✓	
RNF06 upload seguro			✓	
RNF07 anti-robô e limite			✓	
RNF08 edição simultânea		✓	✓	✓
RNF09 tudo ou nada		✓	✓	
RNF10 dado inválido recusado	✓	✓	✓	
RNF11 contratos	✓		✓	
RNF12 painel no celular				✓
RNF13 acessibilidade				✓
RNF14 capacidade		✓	✓	
RNF15 site nunca em branco	✓			✓
RNF16 migrations e CI		✓		

O que os testes encontraram

Problemas reais achados ao escrever esta suíte, todos corrigidos:

achado	onde	correção
Foto usada num texto do site (abertura, rodapé) podia ser apagada. O D1 manda números do JavaScript como REAL, e `CAST(16.0 AS TEXT)` vira `'16.0'`, que não bate com `'16'`.	`functions/api/admin/midias`	comparar convertendo o texto para inteiro: `CAST(valor AS INTEGER) = ?`
Editar só um campo da meta de um projeto dava erro de validação, porque a regra "três campos juntos" rodava sobre o pedido parcial.	`functions/_lib/entidades.ts`	a regra entre campos roda só sobre o registro já mesclado

achado	onde	correção
A auditoria do dia cortava em 500 registros.	`functions/api/admin/auditoria.ts`	limite de 2000 por consulta
Abas do painel com `aria-checked` em `role="tab"` (atributo inválido).	`src/admin/Textos.tsx`	só `aria-selected`
Dicas, horários e botões de apagar do painel abaixo do contraste AA.	`src/admin/admin.css`	tons mais escuros só no painel, o design do site intacto
O mural da diretoria, que rola de lado no celular, não era alcançável pelo teclado.	`Diretoria.tsx`	`tabIndex` e rótulo na lista
Com todos os números ocultos, o site voltava a mostrar todos (achado no uso, virou teste de regressão).	`src/lib/conteudo-remoto.ts`	lista vazia é resposta válida e a seção sai da página
O teste do zip dependia do Python e falhava no Windows sem ele.	`tests/unit/utilitarios.test.ts`	leitura do zip em TypeScript conferida com o CRC-32 do zlib; o Python vira conferência extra opcional

Fora do escopo, por enquanto

- Testes de carga: o tráfego esperado de uma ONG local cabe com folga no plano gratuito (5 milhões de leituras por dia).
- Navegadores além do Chromium: o E2E roda em Chromium no computador e no celular. Firefox e WebKit (Safari) entram adicionando projetos no `playwright.config.ts`.
- Mutação de código (mutation testing) e testes visuais por screenshot.